

Cahier de Conception Frontend

MedConnect ODC — Interface Utilisateur React / TypeScript

Projet	Plateforme de prise de rendez-vous médical
Framework UI	React 18 + TypeScript + Vite
Bibliothèque de style	Tailwind CSS + shadcn/ui
Routing	React Router v6
Gestion des données	TanStack Query (React Query)
Consommation API	Axios

Sommaire

1. Présentation de l'Architecture Frontend
2. Structure de Navigation et Routing
3. Composants Principaux
4. Gestion de l'Authentification (AuthContext + JWT)
5. Couche de Communication API
6. Identité Visuelle et Design System
7. Responsive Design et Accessibilité Mobile
8. Corrections et Améliorations
9. Build et Déploiement Frontend
10. Conclusion

1. Présentation de l'Architecture Frontend

Le frontend de MedConnect ODC est une application monopage (SPA) développée en React 18 avec TypeScript, bundlée via Vite pour des performances de développement et de production optimales. Elle communique exclusivement avec le backend Spring Boot via des appels HTTP REST, en transmettant le jeton JWT dans l'en-tête Authorization de chaque requête authentifiée.

L'architecture s'articule autour de trois principes fondamentaux : la séparation des préoccupations par domaine fonctionnel (auth, patient, médecin, admin), la protection des routes par rôle via un composant gardien dédié (RequireRole), et une expérience utilisateur adaptée aussi bien aux appareils mobiles (via une barre de navigation inférieure) qu'aux postes de travail (via une sidebar latérale).

1.1. Stack technologique détaillée

Technologie	Rôle dans le projet
React 18	Bibliothèque UI principale — composants fonctionnels et hooks.
TypeScript	Typage statique pour fiabiliser les interfaces de données et les props de composants.
Vite	Bundler ultra-rapide — HMR en développement, build optimisé en production.
Tailwind CSS	Utilitaires CSS pour un stylage rapide et cohérent, sans feuille de style globale.
shadcn/ui	Bibliothèque de composants accessibles et personnalisables (Dialog, Toast, Select, etc.).
React Router v6	Gestion déclarative des routes SPA avec protection par rôle.
TanStack Query	Gestion du cache, des états de chargement et des erreurs pour toutes les requêtes serveur.
Axios	Client HTTP configuré avec intercepteurs pour injecter le JWT et centraliser la gestion d'erreurs.

2. Structure de Navigation et Routing

Le routage de l'application est géré par React Router v6. Les routes sont organisées en deux zones distinctes : une zone publique accessible sans authentification, et une zone privée protégée par le composant `RequireRole` qui vérifie le rôle stocké dans le contexte d'authentification avant d'autoriser l'accès.

2.1. Routes publiques

Route	Description
/	Page d'accueil publique — présentation de la plateforme, call-to-action, moteur de recherche simple de médecins.
/login	Connexion patient/médecin + onglet dédié Admin avec champ clé secrète.
/signup	Inscription patient avec formulaire de validation en temps réel.
/reset-password	Réinitialisation du mot de passe par jeton envoyé par email.

2.2. Routes privées — Espace Patient

Route	Description
/patient	Dashboard patient : statistiques personnelles (RDV à venir, consultations passées), actions rapides.
/find-doctors	Recherche de médecins avec filtre par spécialité et réservation de RDV en modal.
/appointments	Liste de tous ses rendez-vous (à venir, passés) avec actions d'annulation.
/appointments/:id	Détail d'un rendez-vous : informations complètes, statut, compte-rendu de consultation si disponible.
/records	Dossier médical : historique complet des consultations et ordonnances.
/settings	Profil, changement de mot de passe, upload de photo de profil.
/messages	Messagerie (version 2 — placeholder intégré dans le layout).

2.3. Routes privées — Espace Médecin

Route	Description
/doctor	Dashboard médecin : statistiques du jour, patients attendus, actions

Route	Description
	rapides.
/appointments	Agenda médecin : liste des rendez-vous par statut, actions de confirmation/annulation.
/appointments/:id	Détail d'un rendez-vous patient : formulaire de saisie du rapport de consultation.
/records	Consultation de l'ensemble des fiches médicales rédigées.
/settings	Profil médecin : spécialité, coordonnées, photo.

2.4. Routes privées — Espace Administrateur

Route	Description
/admin	Dashboard admin : onglets Utilisateurs et Rendez-vous, gestion des comptes, ajout de médecins, supervision globale du planning.

3. Composants Principaux

L'application est structurée en composants réutilisables, chaque composant ayant une responsabilité clairement définie. Voici la liste et la description des composants architecturaux les plus importants.

3.1. Composants de mise en page (Layout)

Composant	Rôle
Logo	Affiche les logos MedConnect ODC et ODC Guinée côte à côte, avec les noms en dessous. Utilisé sur toutes les pages publiques et dans la sidebar.
PublicHeader	En-tête des pages publiques avec les deux logos, les liens de navigation (Accueil, À propos, Connexion, Inscription) et le responsive collapse.
DashboardShell	Wrapper principal du layout privé : intègre la Sidebar, la zone de contenu principale, et la bottom navigation bar mobile.
Sidebar	Navigation latérale des tableaux de bord, adaptée dynamiquement selon le rôle détecté dans le contexte d'authentification (PATIENT / MEDECIN / ADMIN).
DashboardTopbar	Barre supérieure des dashboards : titre de la page active, barre de recherche globale et icône cloche avec compteur de notifications non lues.
Footer	Pied de page commun à toutes les pages publiques.

3.2. Composants d'authentification et de garde

Composant	Rôle
AuthProvider	Contexte React global exposant : user, role, email, signIn(token, role, email) et signOut(). Persiste l'état dans localStorage.
RequireRole	Composant gardien de route : lit le rôle depuis AuthContext, redirige vers /login si l'utilisateur n'est pas authentifié ou si son rôle ne correspond pas à la route. Utilise useNavigate() pour éviter tout rechargement de page.

3.3. Composants fonctionnels — Espace Patient

Composant	Rôle
PatientDashboard	Affiche les statistiques dynamiques (nombre de RDV à venir, consultations passées, médecins consultés). Récupère les données

Composant	Rôle
	via TanStack Query.
FindDoctors	Formulaire de recherche de médecins par spécialité. Ouvre un modal de sélection du créneau et de confirmation de réservation.
AppointmentList	Liste paginée des rendez-vous du patient avec actions d'annulation et indicateurs de statut colorés.
MedicalRecords	Historique des consultations : diagnostic, notes et ordonnances, affichés sous forme de cartes dépliantes.

3.4. Composants fonctionnels — Espace Médecin

Composant	Rôle
DoctorDashboard	Affiche l'agenda du jour, les statistiques (RDV confirmés, en attente, annulés) et les actions rapides de gestion des créneaux.
DisponibiliteManager	Interface de déclaration de nouveaux créneaux (date, heure de début, heure de fin). La durée est calculée automatiquement.
AppointmentsDoctor	Agenda médecin filtrable par statut, avec actions de confirmation, d'annulation et de rédaction du rapport de consultation.
ConsultationForm	Formulaire de saisie du rapport médical (diagnostic, notes, ordonnance) déclenché à la clôture d'un rendez-vous.

3.5. Composants fonctionnels — Espace Administrateur

Composant	Rôle
AdminDashboard	Vue à 360° avec deux onglets : gestion des utilisateurs (création de médecins, activation/désactivation) et supervision des rendez-vous. Gère les statuts EN_ATTENTE et ATTENTE pour la rétrocompatibilité.

3.6. Composants UI transverses

Composant	Rôle
NotificationBell	Icône cloche dans la topbar affichant le compteur de notifications non lues. Au clic, ouvre un panneau déroulant avec la liste des alertes récentes.
Toast / AlertToast	Composant shadcn/ui de notification non-bloquante pour confirmer les actions (réservation, annulation, connexion, erreurs).
ConfirmModal	Modal de confirmation générique utilisé avant toute action irréversible (annulation de RDV, désactivation d'un compte).

Composant	Rôle
StatsCard	Carte de statistiques réutilisable (titre, valeur, icône, tendance) utilisée dans tous les dashboards.
StarRating	Composant d'affichage des étoiles de notation affiché sur les cartes médecin dans la page FindDoctors.
EmptyState	Affichage par défaut avec message et call-to-action lorsqu'une liste est vide (aucun RDV, aucun médecin trouvé, etc.).

4. Gestion de l'Authentification (AuthContext + JWT)

L'authentification repose sur un contexte React global (AuthContext) qui centralise l'état de session et le rend accessible à l'ensemble de l'arbre de composants via le hook `useAuth()`.

4.1. Flux de connexion détaillé

1. L'utilisateur soumet son email et son mot de passe via le formulaire de connexion (+ clé secrète pour l'administrateur).
2. Le frontend envoie une requête POST `/api/auth/login` (ou `/api/auth/admin/login`) via Axios.
3. Le backend vérifie les identifiants (BCrypt), génère un jeton JWT signé incluant le rôle dans ses claims, et le retourne en réponse JSON.
4. Le frontend stocke le token, le rôle et l'email dans `localStorage` via la fonction `signIn()` du contexte.
5. Le composant `RequireRole` lit le rôle depuis `AuthContext` et redirige l'utilisateur vers son dashboard dédié via `useNavigate()`.
6. À chaque requête authentifiée suivante, Axios injecte automatiquement l'en-tête `Authorization: Bearer <token>` via un intercepteur configuré au démarrage.

4.2. Gestion de l'expiration du token

L'intercepteur Axios response détecte les réponses 401 Unauthorized retournées par le backend (jeton expiré ou invalide) et déclenche automatiquement une déconnexion propre : appel de `signOut()`, effacement du `localStorage` et redirection vers `/login`, sans intervention de l'utilisateur.

4.3. Réinitialisation du mot de passe

- L'utilisateur saisit son email sur la page `/login` (onglet 'Mot de passe oublié').
- Le backend génère un jeton à usage unique et l'envoie par email avec un lien vers `/reset-password?token=xxx`.

- L'utilisateur saisit son nouveau mot de passe. Le frontend envoie le token et le nouveau mot de passe au backend pour validation et mise à jour.

5. Couche de Communication API

Toute communication avec le backend est encapsulée dans une couche de service Axios centralisée, organisée par domaine fonctionnel, afin d'éviter la dispersion des appels HTTP dans les composants React.

5.1. Configuration de l'instance Axios

```
// src/api/axiosInstance.ts
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL, // ex: http://localhost:8080/api
  headers: { 'Content-Type': 'application/json' },
});

// Intercepteur de requête : injection automatique du JWT
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Intercepteur de réponse : gestion de l'expiration du token
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.clear();
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```

5.2. Services API par domaine

Service	Fonctions exposées
authService.ts	registerPatient(), loginUser(), loginAdmin(), forgotPassword(), resetPassword()
patientService.ts	getAvailabilities(), bookAppointment(), getMyAppointments(), cancelAppointment(), getMyHistory()
doctorService.ts	createSlot(), getMySlots(), getDoctorAppointments(), confirmAppointment(), cancelAppointment(), createConsultation()
adminService.ts	getAllUsers(), createDoctor(), activateUser(), deactivateUser(), getAllAppointments()
notificationService.ts	getMyNotifications(), markAllAsRead()
userService.ts	getProfile(), updateProfile(), getDoctorDisponibilites(idMedecin)

6. Identité Visuelle et Design System

L'interface de MedConnect ODC respecte les standards visuels d'Orange Digital Center (ODC) tout en incorporant une identité propre à la plateforme médicale.

6.1. Palette de couleurs

Rôle	Couleur	Usage
Primaire	Orange ODC (#E8590C)	Call-to-action, titres principaux, éléments de marque.
Secondaire	Bleu profond (#1B4F72)	Confiance médicale, titres de sections, sidebar active.
Neutre clair	Blanc (#FFFFFF)	Fond des cards, zones de saisie, arrière-plans.
Neutre structurel	Gris léger (#F4F4F4)	Fonds de section, lignes de séparation, états désactivés.
Succès	Vert (#28A745)	Statut CONFIRME, badges positifs.
Alerte	Jaune (#FFC107)	Statut EN_ATTENTE / ATTENTE.
Danger	Rouge (#DC3545)	Statut ANNULE, alertes critiques, boutons de suppression.

6.2. Typographie

- Police principale : Inter ou Poppins — sans-serif, moderne, hautement lisible à toutes les tailles.
- Hiérarchie typographique : H1 (2rem, 700), H2 (1.5rem, 600), H3 (1.25rem, 600), corps (1rem, 400).
- Les titres de section dans les dashboards utilisent le bleu profond ; les accents et liens utilisent l'orange ODC.

6.3. Composants UI récurrents

- Cards : fond blanc, ombre légère (box-shadow), coins arrondis (border-radius 8px), utilisées pour les disponibilités, les rendez-vous et les statistiques.
- Tableaux de bord : grille de statistiques en haut, liste d'actions en bas, avec messages d'état vide (EmptyState) systématiques pour éviter les interfaces incomplètes.
- Modals (Dialog shadow/ui) : confirmation de réservation, saisie de consultation, ajout de médecin par l'admin.
- Toast notifications : confirment les actions réussies (RDV réservé, consultation enregistrée) ou signalent les erreurs (créneau déjà pris, identifiants invalides).

7. Responsive Design et Accessibilité Mobile

La priorité mobile-first est fondamentale pour MedConnect ODC, une majorité des patients guinéens accédant à Internet via leur smartphone.

7.1. Stratégie d'adaptation

Contexte	Comportement
Mobile (< 768 px)	Sidebar masquée, remplacée par une bottom navigation bar fixe avec 4 à 5 onglets icône + libellé. Header simplifié avec menu hamburger.
Tablette (768–1024 px)	Sidebar réduite (icônes seules), contenu principal élargi.
Desktop (> 1024 px)	Sidebar latérale complète avec libellés, topbar avec recherche et notifications.

7.2. Classes Tailwind CSS utilisées pour le responsive

```
// Exemple : DashboardShell
<div className="flex min-h-screen bg-gray-50">
  { /* Sidebar — masquée sur mobile */ }
```

```

<aside className="hidden md:flex md:w-64 flex-col bg-white border-r">
  <Sidebar />
</aside>
{/* Contenu principal */}
<main className="flex-1 p-4 md:p-8 pb-20 md:pb-8">
  <Outlet />
</main>
{/* Bottom nav — visible uniquement sur mobile */}
<nav className="fixed bottom-0 left-0 right-0 bg-white border-t md:hidden">
  <BottomNavBar />
</nav>
</div>

```

8. Corrections et Améliorations

La version 2.0 du frontend a apporté un ensemble de corrections critiques et d'améliorations fonctionnelles issues des retours de tests :

Type	Catégorie	Description
Bug critique	API	Harmonisation de <code>getConsultations()</code> → <code>getMyConsultations()</code> sur l'ensemble des composants concernés.
Bug UI	Routing	<code>RequireRole</code> utilise désormais <code>useNavigate()</code> au lieu de <code>window.location.href</code> , éliminant les rechargements complets de page.
Bug UI	Build	Remplacement de l'import dynamique de <code>Login.tsx</code> par un import statique (suppression du warning Vite).
Bug API	Patient	<code>FindDoctors</code> : utilise <code>getDoctorDisponibilites(idMedecin)</code> au lieu d'un filtre côté client pour éviter les incohérences de données.
Bug Admin	Statuts	<code>AdminDashboard</code> gère correctement les deux valeurs <code>EN_ATTENTE</code> et <code>ATTENTE</code> pour la rétrocompatibilité.
Amélioration	Admin	<code>AdminDashboard</code> : interface en onglets Utilisateurs / Rendez-vous pour une meilleure lisibilité.
Amélioration	Patient	<code>Sidebar</code> : lien 'Trouver un Médecin' ajouté dans le menu patient.
Amélioration	Mobile	<code>DashboardShell</code> : bottom navigation bar pour mobile entièrement fonctionnelle.

Type	Catégorie	Description
Amélioration	Data	PatientDashboard / DoctorDashboard : statistiques dynamiques issues de l'API (plus de valeurs statiques codées en dur).
Branding	Identité	Logo MedConnect ODC + Logo ODC côte à côte avec noms en dessous, présents sur toutes les pages.
UX	Interface	Étoiles de notation affichées sur les cartes médecins dans FindDoctors.
UX	Interface	Composant EmptyState avec CTA intégré dans tous les tableaux et listes.

9. Build et Déploiement Frontend

- Build : `npm run build` → génère le dossier `dist/` contenant les assets optimisés et minifiés.
- Hébergement recommandé : Vercel, Netlify, ou serveur Nginx configuré pour le mode SPA (redirection 404 → `index.html`).
- Variable d'environnement requise : `VITE_API_BASE_URL=https://votre-api.com/api` (fichier `.env.production`).
- Les variables préfixées `VITE_` sont exposées au bundle client — ne jamais y mettre de secrets (clés API privées, mots de passe).

10. Conclusion

Le frontend de MedConnect ODC offre une expérience utilisateur fluide, accessible et adaptée aux trois rôles de la plateforme. L'architecture basée sur React 18, TypeScript, TanStack Query et un système de contexte d'authentification centralisé garantit une base de code maintenable, fortement typée et prête à accueillir les évolutions fonctionnelles de la version 3 (messagerie, téléconsultation, paiement en ligne).